



Eisdielen San Remo --> BestellAbwicklung

Browser-basierte Touch-Kasse für die Eisdielen San Remo, geschrieben in Delphi 12.1 mit TMS Web Core (Trial v2.4.6.1, kompiliert mit pas2js 2.3.1). Läuft im Browser auf Windows-Rechnern, Android-Tablets und auch auf dem Handy (im Querformat). Wird über GitHub Pages ausgeliefert.

Begleitend dazu gibt es einen **Artikel-Editor** als eigenständige React-HTML-Anwendung, mit dem die Artikelliste komfortabel gepflegt und per Knopfdruck direkt nach GitHub gepusht werden kann.

Live-URLs

Anwendung	URL
Kasse	https://rwhexa.github.io/eisdielezw/EisdieleKasse.html
Artikel-Editor	https://rwhexa.github.io/eisdielezw/artikel-editor.html

Repository: <https://github.com/rwhexa/eisdielezw>

Versionshistorie

Version	Stand	Hauptänderungen
v0.1	erste Vorführung beim Eisdielen-Inhaber	Touch-Kasse, Tisch-Verwaltung, MwSt-Logik, Pastell-Design, RW-Logo
v0.2	Absturzsicherung	localStorage-Persistenz aller offenen Tische
v0.3	aktuelle Version	Artikelliste extern in produkte.json , externer Editor mit GitHub-API-Anbindung

Funktionsumfang (aktueller Stand 24 Mai 2026)

Bestellungen-Kasse

- **Tisch-Verwaltung** für Tisch 1–8, Theke und Mitnahme
- **Eigener Warenkorb pro Tisch**, der beim Tischwechsel automatisch gespeichert wird – Nachbestellungen addieren sich zur bestehenden Bestellung
- **Tisch-Leiste** oben zeigt jederzeit alle offenen Tische mit ihrer Summe; aktive Tische in Koralle, belegte Tische in Mint
- **MwSt-Logik**: Tische 1–8 standardmäßig Vor Ort (19 %), Theke und Mitnahme standardmäßig zum Mitnehmen (7 %); pro Tisch umschaltbar

- **Artikel-Kacheln** in vier Kategorien (frei änderbar): Eis, Becher, Waffel, Getränk
- **Netto/MwSt/Brutto-Splittung** im Warenkorb
- **Storno-** und **Bar-Button** nebeneinander unter den Beträgen
- **Eisdielen-Pastell-Farbschema** mit Koralle/Mint/Crémé
- **RW-Logo** und **Versions-Anzeige** in der Topbar
- **localStorage-Persistenz** aller offenen Tische als Absturz-Sicherung
- **Artikelliste extern als JSON** – Pflege ohne Delphi-Compile

Artikel-Editor

- **React-basierte HTML-Anwendung** in einer einzigen Datei (alles inline)
- **JSON-Datei laden** per Drag & Drop oder Dateiauswahl
- **Tabellarische Bearbeitung** mit Sortierung nach Kategorie, Filterung, Inline-Edit
- **Emoji-Picker** für die Icon-Auswahl bei Artikeln
- **Download als JSON** für Backup oder manuellen Upload
- **GitHub-API-Push** mit einem Klick: Datei wird direkt ins Repo committed, ohne Download-Umweg
- **RW-Logo** im Header
- **Token-gesicherte** Verbindung zu GitHub (Personal Access Token)

Projektstruktur

EisdielenKasse/	
├─ EisdielenKasse.dpr	Projektdatei
├─ EisdielenKasse.html	Project-HTML mit CSS, Boot-Wrapper, PWA-Hooks
├─ UMain.pas	Main-Unit mit Datenmodell und Form-Aufbau
├─ UMain.dfm	Form-Definition (epIgnore, efCSS)
├─ UMain.html	Platzhalter (leer, aber von TMS referenziert)
├─ produkte.json	Artikelliste (Pflege durch Eisdielen-Inhaber)
├─ artikel-editor.html	React-basierter Artikel-Editor (selbstgenügend)
├─ logorw.png	RW-Logo mit transparentem Hintergrund (256x256)
├─ index.html	Redirect-Stub für Aufruf der Repo-Root-URL
├─ manifest.json	PWA-Manifest (für späteren PWA-Ausbau bereit)
├─ service-worker.js	Service Worker (für späteren PWA-Ausbau bereit)
└─ README.md	Dieses Dokument

Im Output-Verzeichnis **TMSWeb\Release** landet nach dem Compile:

- **EisdielenKasse.html** – gerenderte HTML
- **EisdielenKasse.js** – kompilierte JS, ca. 2,4 MB

Wichtig: **logorw.png**, **produkte.json**, **manifest.json**, **service-worker.js** und **artikel-editor.html** müssen **manuell** nach **TMSWeb\Release** kopiert werden, da TMS Web Core sie beim Build nicht automatisch übernimmt.

Gleiches gilt für `UMain.html` – die wird vom kompilierten JS zur Laufzeit referenziert.

Datenmodell

```
TProduct
├─ Id, Cat, Nm (Name), Pr (Preis brutto), Ico (Emoji)
└─ Vat19: Boolean (true = immer 19 %, z. B. Getränke)

TOrderItem
├─ Product: TProduct
└─ Qty: Integer

TTable
├─ Id, Name, Mode (cmTakeAway / cmEatHere)
└─ Items: array of TOrderItem
```

`TForm1` hält dynamische Arrays `FProducts`, `FTables` und einen Zeiger `FActiveTable` auf den gerade aktiven Tisch.

Artikelliste in produkte.json

Die Artikelliste wird beim Start der Kasse asynchron aus `produkte.json` geladen. Format:

```
{
  "artikel": [
    { "id": 1, "kategorie": "Eis", "name": "Vanille", "preis": 1.50,
      "icon": "🍦" },
    { "id": 30, "kategorie": "Getränk", "name": "Espresso", "preis": 2.20,
      "icon": "☕" }
  ]
}
```

Felder pro Artikel:

Feld	Typ	Pflicht	Bemerkung
<code>id</code>	Zahl	ja	eindeutig, darf nicht doppelt sein. Bestehende IDs nicht ändern – sonst verlieren noch offene Tische den Bezug zum Artikel
<code>kategorie</code>	Text	ja	bestimmt automatisch die Tabs; Reihenfolge des ersten Auftretens = Tab-Reihenfolge
<code>name</code>	Text	ja	wird im Tile und im Warenkorb angezeigt

Feld	Typ	Pflicht	Bemerkung
preis	Zahl	empfohlen	mit Punkt, nicht Komma (1.50, nicht 1,50). Fehlt das Feld, wird 0 angenommen
icon	Text	optional	ein Emoji als Symbol
mwst19	Boolean	optional	true für immer-19%-Artikel. Aktuell wird das Feld vom Editor nicht geschrieben → alle Artikel folgen dem Tisch-Modus (siehe MwSt-Hinweis weiter unten)

Robustheit:

- Datei fehlt, ist leer oder kaputt → die Kasse fällt automatisch auf die eingebaute Standardliste (in **SeedProducts**) zurück und zeigt einen Hinweis
- Pflichtfeld fehlt bei einem Artikel → dieser Artikel wird übersprungen, die anderen werden geladen
- Im localStorage gespeicherte Bestellungen mit einer Produkt-ID, die nicht mehr in der neuen Liste vorkommt, werden beim Laden übersprungen (mit Hinweis)

Artikel-Editor

Der Artikel-Editor ist eine eigenständige HTML-Datei (React + Babel via CDN, RW-Logo inline als Base64). Er kann sowohl von GitHub Pages als auch lokal per Doppelklick geöffnet werden.

Erstmaliges Token-Setup

Damit der Editor direkt nach GitHub speichern kann, wird einmalig ein **Personal Access Token (Fine-grained)** angelegt:

1. Auf GitHub einloggen → Profilbild → **Settings**
2. Links unten ganz scrollen → **Developer settings**
3. **Personal access tokens** → **Fine-grained tokens**
4. **Generate new token**
5. Felder ausfüllen:
 - **Token name:** "Artikel-Editor"
 - **Expiration:** ein Jahr oder länger
 - **Repository access:** *Only select repositories* → **eisdielerw**
 - **Permissions** → **Repository permissions** → **Contents:** *Read and write*
6. **Generate token** klicken
7. Token sofort kopieren (er wird nur einmal angezeigt)
8. Im Editor das Zahnrad-Symbol (⚙️) anklicken
9. Token einfügen → "Verbindung testen" → "Speichern"

Der Token wird im **localStorage** des Browsers gespeichert. Bei Verlust des Geräts kann er in GitHub mit einem Klick widerrufen werden.

Editier-Workflow

1. Editor öffnen (Lokal oder über die GitHub-Pages-URL)
2. ↑ **Laden** – aktuelle `produkte.json` einlesen (Drag & Drop oder Dateipicker)
3. Artikel ändern, hinzufügen, löschen, Kategorie zuweisen, Emoji wählen
4.  **GitHub** klicken → Datei wird direkt ins Repo committed
5. 1–2 Minuten warten (GitHub Pages-Cache)
6. Auf der Kasse-URL Strg+F5 – die neuen Preise sind aktiv

Der Editor unterscheidet sich nach Aufruf-Ort beim Token-Speicher: ein Token, der lokal eingegeben wurde, gilt nicht auf der GitHub-Pages-Version und umgekehrt (Browser trennen `file://` und `https://` strikt). Falls der Editor an mehreren Stellen genutzt wird, muss der Token an jeder Stelle einmal eingegeben werden.

Manueller Pflege-Workflow als Backup

Sollte der Editor mal nicht funktionieren (Token abgelaufen, Internet weg etc.), kann die `produkte.json` weiterhin direkt im GitHub-Browser-Editor bearbeitet werden – Stift-Symbol an der Datei, ändern, Commit changes.

MwSt-Hinweis (für späteren Echtbetrieb)

Aktuell folgen **alle** Artikel dem Tisch-Modus:

- Tische 1–8 (Vor Ort): 19 % MwSt
- Theke und Mitnahme (Takeaway): 7 % MwSt

In Deutschland sind **Getränke** allerdings steuerlich immer mit 19 % zu erfassen, unabhängig vom Verzehr-Ort. Für die Demo-/Testphase ohne TSE-Anbindung ist die aktuelle Vereinfachung in Ordnung, für den Echtbetrieb muss das angepasst werden.

Die Lösung ist eine 3-Zeilen-Änderung in `ParseProductsJson` in `UMain.pas`:

```
// Vorher:
Pmwst := False;
if ItemObj.hasOwnProperty('mwst19') then
  Pmwst := Boolean(ItemObj['mwst19']);

// Nachher:
Pmwst := SameText(Pkat, 'Getränk'); // Default: Getraenke immer 19%
if ItemObj.hasOwnProperty('mwst19') then
  Pmwst := Boolean(ItemObj['mwst19']); // optionaler Override pro Artikel
```

Damit braucht die JSON kein `mwst19`-Feld – der Pascal-Code leitet die Sonderbehandlung automatisch aus der Kategorie ab.

localStorage-Persistenz

Der komplette Tisch-Zustand wird nach jeder Bestelländerung in den localStorage geschrieben (`StoreTablesState`) und beim App-Start wieder geladen (`RestoreTablesState`). Speicher-Key: `sanremo_state_v1`.

Was gespeichert wird:

- Pro Tisch: ID, Verzehr-Modus
- Pro Artikel: nur **Produkt-ID + Menge** (nicht Name und Preis)

Beim Laden werden die Produkt-IDs wieder mit den echten Produkten verknüpft. Dadurch wirken Preisänderungen in `produkte.json` auch auf bereits offene Tische.

Robustheit:

- localStorage nicht verfügbar (Speicher voll etc.) → App läuft normal weiter, nur ohne Sicherung
- Korrupter Speicherinhalt → sauberer Neustart mit leeren Tischen
- Produkt-ID nicht mehr auffindbar → Artikel wird übersprungen mit kurzem Hinweis
- Speicher-Key versioniert (`_v1`) – bei künftiger Formatänderung einfach `_v2` setzen, alter Stand wird automatisch ignoriert

Bekannte Stolperfallen mit TMS Web Core Trial 2.4.6.1

Diese Punkte sind in der Trial-Version zu beheben. Bei einer Vollversion oder neueren Releases können die Workarounds schrittweise zurückgebaut werden.

1. pas2js Trial vergisst den rtl.run-Boot-Aufruf

Der pas2js-Output endet mit `});` und ohne den eigentlich erwarteten `rtl.run('program');`-Aufruf. Folge: JS lädt, wird aber nicht gestartet, Browser bleibt leer.

Workaround in `EisdieleKasse.html`:

```
<script src="$(ProjectName).js"></script>
<script>
  if (typeof rtl !== 'undefined' && typeof rtl.run === 'function') {
    rtl.run('program');
  } else {
    alert('FEHLER: rtl ist nicht definiert.');
```

```
}  
</script>
```

2. Generics überfordern den Trial-Compiler

`TObjectList<T>` mit drei verschiedenen Typparametern bricht den JS-Output mittendrin ab. Workaround: dynamische Arrays:

```
FProducts: array of TProduct;  
FTables  : array of TTable;
```

Plus eigene Helper-Methoden `AddProduct`, `AddTable`, `FreeProducts`, `FreeTables`.

3. ElementPosition und ElementFont müssen explizit gesetzt werden

Default: Komponenten bekommen Inline-Styles mit `position: absolute; width: 100px; height: 25px` mitgegeben, die CSS-Klassen überschreiben.

Workaround auf jedem programmatisch erzeugten Control:

```
Btn := TWebButton.Create(Self);  
Btn.ElementPosition := epIgnore;  
Btn.ElementFont := efCSS;  
Btn.Parent := pnlTop;  
Btn.ElementClassName := 'mode-btn';
```

In der DFM analog:

```
ElementClassName = 'host'  
ElementFont = efCSS  
ElementPosition = epIgnore
```

4. TMS injiziert anonyme div-Wrapper

Zwischen einem `<span class="kasse-root">` und seinen Kindern liegt ein unstyled `<div>`. Grid- und Flex-Layouts der Kinder brechen zusammen.

Workaround in CSS:

```
.kasse-root > div, .main > div, .cart > div, .totals > div /* etc. */ {  
    display: contents !important;
```

```
}
```

5. Inline-Styles brauchen !important-Override

TMS gibt trotz `epIgnore` weiter inline `width: 100px; height: 25px` mit.

Workaround in CSS: Container-Klassen mit `!important` auf `width: auto`, `height: auto` und gewünschten `display`-Modus zwingen.

6. Beim Deployment alles aus TMSWeb\Release committen

Die kompilierte JS-Datei referenziert intern `UMain.html`. Wird diese nicht mitgepusht, kommt eine 404-Antwort vom GitHub-Pages-Server, die sich in der Anzeige mit der Kasse mischt.

Gleiches gilt für `produkte.json`, `logorw.png`, `artikel-editor.html` und (bei PWA-Aktivierung später) `manifest.json` und `service-worker.js` – TMS kopiert diese nicht automatisch nach `TMSWeb\Release`. Manueller Schritt.

7. Geerbte Methoden nicht überschreiben

`TWebForm` erbt von `TControl`, das schon `SaveState/LoadState` besitzt. Deshalb wurden die Persistenz-Methoden bewusst `StoreTablesState/RestoreTablesState` benannt.

Build und Start

1. Projekt in Delphi 12.1 öffnen
2. Build-Konfiguration auf **Release** stellen (Projekt-Manager → Build Configurations → Rechtsklick auf Release → Aktivieren)
3. **Projekt** → **Neu erstellen** (Strg+F9 reicht nicht für saubere HTML-Übernahme)
4. F9 zum Starten – TMS Web Core öffnet den Browser automatisch über den eingebauten Dev-Server auf `http://localhost:8000/`
5. Bei Änderungen: Strg+F5 im Browser für Hard-Reload (Cache umgehen)

Deployment auf GitHub Pages

Diese Dateien müssen im Repo `eisdielerw` liegen:

Datei	bei welchem Anlass aktualisieren
<code>EisdieleKasse.js</code>	bei jeder Code-Änderung in der Kasse (aus <code>TMSWeb\Release</code>)
<code>EisdieleKasse.html</code>	nur bei CSS- oder PWA-Änderungen
<code>UMain.html</code>	nur einmal, ändert sich nicht

Datei	bei welchem Anlass aktualisieren
produkte.json	wird vom Editor automatisch gepusht; manuell nur bei Bedarf
artikel-editor.html	bei Editor-Erweiterungen
logorw.png	nur bei Logo-Wechsel
index.html	nur einmal als Redirect-Stub
manifest.json	bei PWA-Aktivierung
service-worker.js	bei PWA-Aktivierung

Nach dem Push: GitHub Pages liefert die Inhalte innerhalb 1–2 Minuten unter den Live-URLs aus.

Betrieb auf Android-Tablet

Empfohlene Browser-App: **Fully Kiosk Browser** (Vollbild ohne Adresszeile, gegen versehentliches Verlassen sperrbar, Auto-Start).

Wichtige Einstellungen:

- **Start URL** auf die Kasse-URL setzen
- **Keep Screen On** an
- **Kiosk Mode** mit PIN aktivieren
- **Auto-Confirm JS Dialogs** an (umgeht Trial-Nag-Dialog)
- **Start on Boot** an (Kasse kommt nach Tablet-Einschalten automatisch)

Geplante nächste Schritte

Modul	Aufwand	Beschreibung
MwSt-Korrektur für Getränke	sehr klein	3-Zeilen-Änderung in <code>ParseProductsJson</code> , dann sind Getränke automatisch 19 % unabhängig vom Tisch-Modus. Wird vor Echtbetrieb gemacht
PWA-Aktivierung	klein	Manifest und Service Worker sind vorhanden, müssen nur committed und in der HTML verlinkt werden. Home-Screen-Installation + Offline-Betrieb
Tagesabschluss / Z-Bericht	mittel	Verkaufsliste mit CSV-Export für den Steuerberater
Bondruck	mittel	ESC/POS via Netzwerkdrucker (z. B. Epson TM-T20), HTTP-Brücke als Helper
TSE-Anbindung	groß	Pflicht im Echtbetrieb (Kassensicherungsverordnung). Cloud-TSE (fiskaly, Deutsche Fiskal) per REST oder Hardware-TSE (Swissbit USB)

Modul	Aufwand	Beschreibung
Mehrere Kassen synchron	groß	REST-Backend (Supabase, eigene API) oder MQTT-Sync

Versionsinfo

- Delphi 12.1 Athens
- TMS Web Core Trial v2.4.6.1
- pas2js 2.3.1 (Build 2023-11-19)
- App-Version Kasse: v0.3 (sichtbar in der Topbar)
- Artikel-Editor: React 18 + Babel Standalone via CDN
- Getestet in Chrome (Windows) und Fully Kiosk Browser (Android-Tablet 7")

TSE-Hinweis für Echtbetrieb

Sobald die Software echte Verkäufe abwickelt, schreibt die deutsche Kassensicherungsverordnung eine zertifizierte TSE vor. Ohne TSE darf die Kasse nicht eingesetzt werden. Optionen:

- **Cloud-TSE** (fiskaly, Deutsche Fiskal): einfache HTTPS-Anbindung, ca. 10–15 € pro Monat
- **Hardware-TSE** (Swissbit USB, Epson TSE-Drucker): einmalig teurer, funktioniert offline

Der Bon muss zusätzlich QR-Code, Signatur, Transaktionszähler und Seriennummer der TSE enthalten – das übernimmt die TSE-API.

Sicherheits-Hinweis zum Personal Access Token

Der für den Artikel-Editor verwendete Token wird im `localStorage` des Browsers gespeichert. Auf einem Tablet im Eisdielen-Betrieb hinter dem Tresen ist das praktikabel. Folgendes ist trotzdem zu beachten:

- Token mit **minimalen Berechtigungen** ausstatten: nur das Repo `eisdielerw`, nur `Contents: write`
- **Ablaufdatum setzen** (z. B. 1 Jahr) – Token läuft dann automatisch ab
- Bei **Verlust des Geräts**: Token in GitHub sofort widerrufen (Settings → Developer settings → Tokens → Revoke)
- Token **niemals in Code committen**, niemals in Screenshots
- Bei Wechsel des Personals den Token erneuern